

Which Layer Is the Embedding? Frozen Tabular and Relational Foundation Models as Entity Encoders on Real Relational Data

Yarin Benizri

Hebrew University of Jerusalem · experiments on a Slurm cluster with H200 GPUs · code: frozen-embeddings-retrieval · draft, 3 October 2026 · companion paper: *Open TabFM-Auto*

ABSTRACT. Tabular foundation models predict in context, and their hidden states are now used as row and entity embeddings, nearly always taken from the last layer. We ask which layer is the embedding. On 8 entity-classification tasks from RelBench, with official splits and evaluator, we probe every layer of seven frozen models: TabPFN v2, TabPFN-2.5, TabICLv2, Kumo Tabular S/M/L, and the graph-based Kumo Relational, which reads the database itself. Each model is given four kinds of in-context labels: the real labels, random labels, k-means clusters, or zeros. (1) The last layer is rarely the best one. For Kumo Relational a linear probe on its early graph layer beats the model’s own prediction on 7 of 8 tasks (mean AUROC 0.748 vs 0.680); on one task the head is at chance (0.49) while that layer reaches 0.79. For TabPFN with real labels, blocks 8–9 are picked on 5 of 8 tasks, in line with mechanistic work. (2) Without real labels, frozen embeddings are worth their input, not more: Kumo Relational with random labels matches hand-built relational features (0.728 vs 0.727) with no feature engineering, while TabPFN over those features falls below them. (3) A label-free statistic, the intrinsic dimension of a layer’s output, picks a layer nearly as well as validation labels (mean regret 0.020 vs 0.019; last layer 0.036). (4) Models agree at their last layers (cross-model CKA 0.88) far more than at their best layers (0.71): the last layers converge on the label, the useful layers do not. (5) A protocol trap: if probe-training rows share time periods with the context rows, real-label late layers look useless (last layer 0.665 vs 0.762 with a time split). As a retrieval test, no frozen embedding is a recommendation embedding on rel-hm.

1. Introduction

A tabular foundation model (TFM) reads labelled rows as a prompt and predicts new rows in one forward pass. Because nothing is fitted, its hidden states are attractive as embeddings: one frozen model, any table, no training. Benchmarks of tabular embeddings (TEmBed, 2026) and practitioners take the last layer, usually via `get_embeddings()`. But the last layer of an in-context model is shaped by the in-context labels, and mechanistic studies find that the label is read out at a particular depth. The question of which layer gives the best entity embedding, on real relational data, has not been asked.

Contributions. (i) A layer-by-layer probe of seven open TFMs, including a relational one, on 8 RelBench tasks with official evaluation. (ii) The effect of the in-context labels (real, random, k-means, zeros) on what each layer keeps. (iii) A test of whether label-free geometry (effective rank, TwoNN intrinsic dimension, anisotropy, k-means silhouette) can pick the layer. (iv) Layer similarity within and across models by linear CKA. (v) A protocol finding for layer probes of in-context models. (vi) A retrieval test of frozen embeddings on rel-hm recommendation.

2. Related work

Layers of tabular foundation models. “Is One Layer Enough?” (ICML 2026) compares layers of six TFMs by CKA and cosine similarity and finds strong depth redundancy. “A Mechanistic Study of Tabular Foundation Models” (2026) finds that a probe for the target in TabPFN v2 jumps between layers 8 and 9, while TabICLv2 is readable early. Both study prediction on single tables; we study embeddings for entities in relational databases.

Tabular embeddings. TEmBed (IBM, 2026) benchmarks row, column and table embeddings; TFM rows enter as last-layer `get_embeddings()` and are weak on text-heavy tables. Relational foundation models (KumoRFM, OpenRFM) predict over database subgraphs; Kumo Relational (NVIDIA, 2026) is an open one.

Probing and geometry. Linear probes (Alain & Bengio, 2017) read what a layer encodes; CKA (Kornblith et al., 2019) compares representations; effective rank (RankMe, Garrido et al., 2023), TwoNN intrinsic dimension (Facco et al., 2017) and anisotropy (Ethayarajh, 2019) describe their shape without labels.

3. Setup

Tasks. 8 RelBench entity classification tasks: rel-f1 driver-dnf and driver-top3, rel-trial study-outcome, rel-event user-repeat and user-ignore, rel-avito user-visits and user-clicks, rel-hm user-churn. Official train/val/test tables; test AUROC by the official evaluator.

Inputs. For each task row (entity, time) we build a temporally safe 2-hop subgraph: the entity row, its latest 50 child rows of each kind with time $\leq t$, and its parent rows. Kumo Relational reads the subgraph directly. The tabular models read a flattened version: the entity's columns plus, per child table, the count, the days since the last event and the means of numeric columns.

Context, probes, selection. Each model is given a context of training rows with one of four label kinds: real labels, random labels, k-means cluster ids of the features, or zeros. We record every layer's output for the query rows: the row

encoder, the graph layer (Kumo Relational), each transformer block, and the final norm. A logistic-regression probe ($C = 0.1$, standardised inputs) and a cosine kNN probe ($k = 50$) are trained on 4,000 training rows. The layer is picked by validation AUROC; *regret* is the gap to the best test layer. *Raw* is the same probe on the flattened features.

Time split. The probe-training rows are always strictly later than every context row, as the validation and test rows are (Section 4.5 shows why this matters).

Models. Kumo Relational (study J18, 8 tasks, 512 context entities); TabPFN v2 (J19, 8 tasks, 3,000 context rows, four label kinds); TabPFN v2, TabPFN-2.5, TabICLv2 and Kumo Tabular S/M/L with label-free geometry and CKA (J21, 3 of 5 tasks so far: f1 dnf, f1 top3, trial outcome).

4. Results

Table 1. Real labels in context, test AUROC. *own*: the model's own prediction; *last*: probe on the last layer; *val-picked*: probe on the layer with the best validation AUROC (layer in grey); *random ctx*: Kumo Relational's val-picked layer with random labels in context. Bold: best of the model's three columns.

task	raw features	Kumo Relational (reads the database)				TabPFN v2 (flattened features)		
		own	last	val-picked	random ctx	own	last	val-picked
avito clicks	0.623	0.639	0.607	0.635 graph	0.632	0.629	0.627	0.634 block 8
avito visits	0.649	0.611	0.640	0.648 ICL 0	0.655	0.648	0.656	0.656 block 8
event ignore	0.883	0.838	0.861	0.877 graph	0.850	0.892	0.901	0.893 block 9
event repeat	0.674	0.493	0.766	0.786 ICL 2	0.673	0.708	0.720	0.733 block 8
f1 dnf	0.790	0.769	0.781	0.778 ICL 0	0.773	0.796	0.775	0.824 block 8
f1 top3	0.891	0.859	0.785	0.875 graph	0.849	0.885	0.904	0.899 block 0
hm churn	0.678	0.608	0.667	0.680 ICL 1	0.682	0.681	0.678	0.679 block 0
trial outcome	0.630	0.625	0.667	0.709 row enc.	0.708	0.689	0.702	0.667 block 5
mean (8 tasks)	0.727	0.680	0.722	0.748	0.728	0.741	0.745	0.748

4.1 The last layer is rarely the best one

Kumo Relational: read the graph layer, not the head. The val-picked layer is the graph layer or one of the first transformer layers on 7 of 8 tasks. It beats the model's own prediction on 7 of 8 tasks (mean 0.748 vs 0.680) and the last layer on most (mean 0.722). Later in-context layers lose linear signal step by step (Figure 1). On rel-event user-repeat the model's head is at chance

(0.49) on probe-training and test rows alike; a direct check of its per-class outputs confirms this is the model, not our score extraction. The graph layer of the same forward pass reaches 0.79.

TabPFN: blocks 8-9. With real labels, the val-picked block is 8 or 9 on 5 of 8 tasks, which is where the mechanistic study locates TabPFN's label read-out. The gain over the model's own prediction is small (mean 0.748 vs 0.741; last layer 0.745).

Which layer holds the signal? Linear probe on each layer, minus raw features

Test AUROC gain over a probe on the raw relational features (0 = no better). Mean over RelBench tasks. x: relative depth.

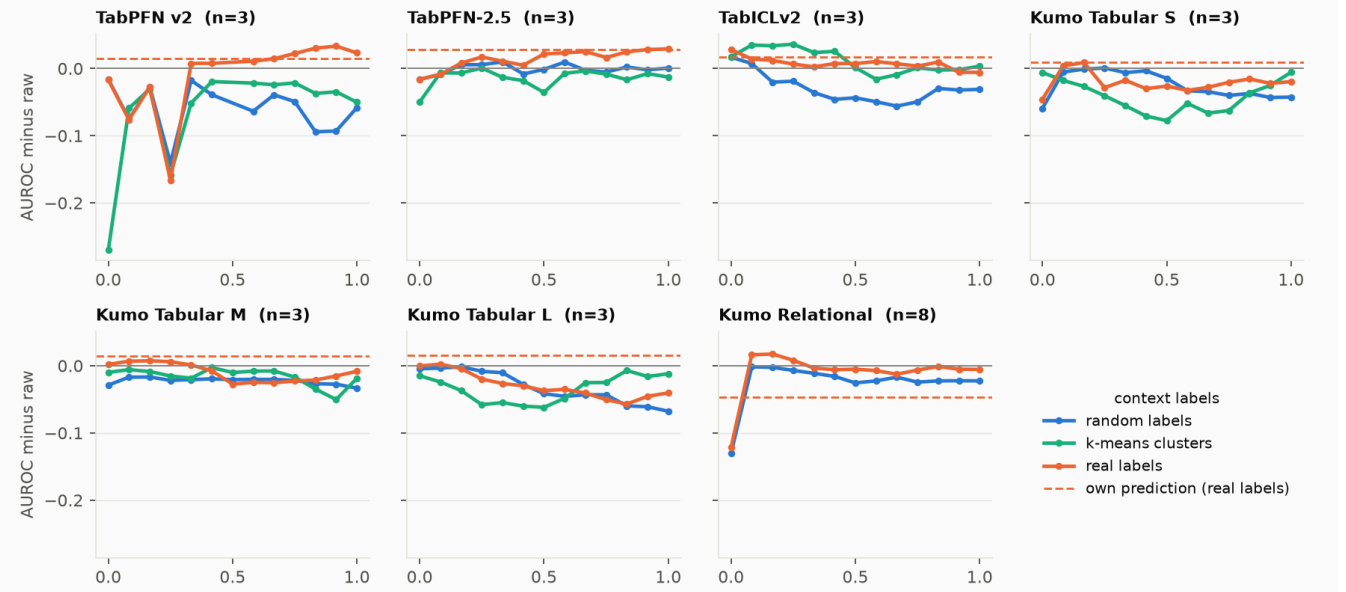


Figure 1. Probe AUROC minus the raw-feature probe at each relative depth, mean over tasks, one panel per model, one line per kind of in-context label. Dashed: the model's own prediction with real labels.

4.2 Without real labels, embeddings are worth their input

Table 2. Mean test AUROC of the val-picked layer by kind of in-context label, and the model's own prediction (real labels).

model	tasks	raw	zeros	random	k-means	real labels	own pred.
Kumo Relational	8	0.727	—	0.728	—	0.748	0.680
TabPFN v2 (J19)	8	0.727	0.704	0.717	0.701	0.748	0.741
TabPFN v2	3	0.770	—	0.774	0.735	0.797	0.790
TabPFN-2.5	3	0.770	—	0.775	0.766	0.801	0.798
TabICLv2	3	0.770	—	0.753	0.789	0.796	0.787
Kumo Tabular S	3	0.770	—	0.717	0.778	0.727	0.780
Kumo Tabular M	3	0.770	—	0.761	0.790	0.783	0.785
Kumo Tabular L	3	0.770	—	0.763	0.774	0.766	0.786

Kumo Relational with random labels matches a probe on hand-built relational features (0.728 vs 0.727) while reading the database with no feature engineering; it is above them on 4 of 8 tasks. TabPFN over the same features falls below them with every label-free context, and the k-means context, which recovers planted segments

on synthetic data, does not help on real tasks. The tabular models in J21 behave alike: label-free layers are close to the raw features on average, with rel-trial the exception, where the early layers of TabICLv2 and Kumo Tabular beat them by 0.06–0.08.

4.3 Can label-free geometry pick the layer?

Table 3. Regret (best test layer minus chosen layer, AUROC) of each way to choose a layer, over 54 model $\times$ task $\times$ context runs (J21). Geometry: choose the layer with the largest or smallest value; which one is decided leave-one-task-out. ρ : mean Spearman correlation of the statistic with layer test AUROC.

layer chosen by	mean regret	median	Spearman ρ
validation labels	0.019	0.006	—
intrinsic dimension (TwoNN)	0.020	0.011	-0.19
last layer	0.036	0.028	—
anisotropy (mean cosine)	0.038	0.023	-0.10
average layer	0.039	0.032	—
effective rank	0.042	0.011	-0.36
k-means silhouette	0.045	0.026	-0.06

Validation labels are the best guide (mean regret 0.019), but the TwoNN intrinsic dimension is nearly as good (0.020) without any label, and better than taking the last layer (0.036). Effective rank is the most consistent signal ($\rho =$

-0.36, negative in 80 % of runs): layers whose outputs spread over more directions are worse probes. Its mean regret (0.042) is hurt by a few large misses.

Label-free geometry of each layer (real-label context)

Mean over RelBench tasks. Columns: relative depth, first layer (0) to last (1). Darker = larger within that model's row.

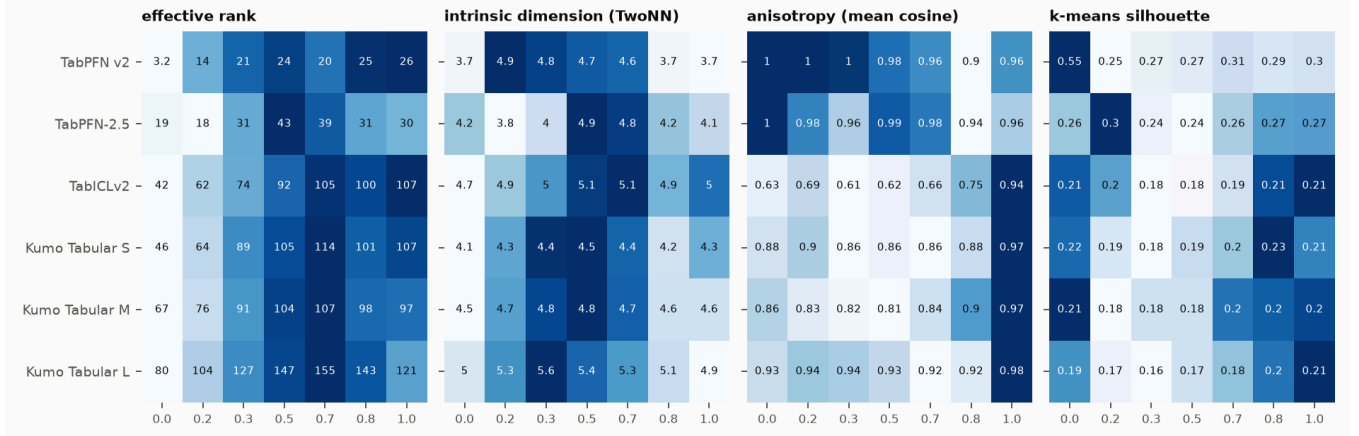


Figure 2. Label-free geometry of each layer, real labels in context, mean over tasks; colour within each model's row.

4.4 Models agree at the last layer, not at the best one

Table 4. Mean linear CKA between models (J21, real labels, test rows), at each model's val-picked layer and at its last layer.

	all model pairs	TabICL/Kumo family	vs raw features
best layer	0.71	0.89	0.47
last layer	0.88	0.92	0.55

The last layers of different models are similar to each other (CKA 0.88): with real labels in context they converge on a representation of the label. The val-picked layers are less alike (0.71), except within the TabICL/Kumo Tabular family, which share an architecture. Both are far from the raw features (0.47 and 0.55). The useful layer keeps model-specific structure that the last layer has already projected away (Figure 3).

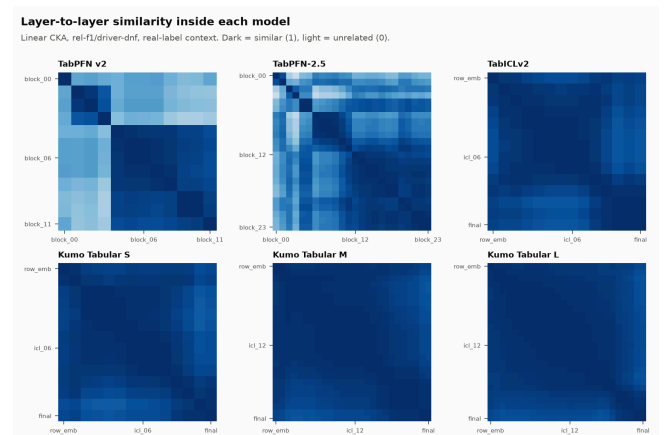


Figure 3. Layer-by-layer CKA inside each model on one task. Blocks of dark cells are layers that compute nearly the same representation.

4.5 A protocol trap for layer probes of in-context models

Our first runs drew the context rows and the probe-training rows at random from the same training table. Most probe-training rows then

had their own entity in the context at a neighbouring date. With real labels, the late layers carry a copy of that neighbour's label, which validation and test rows, later than every context row, never get. A linear probe learns the copy and fails: two models' last layers scored 0.40 and 0.43, below chance, while a kNN probe on the same layers stayed at 0.75–0.79. The time split removes it:

Table 5. The same runs under both splits, mean test AUROC.

context	split	runs	last, linear	last, kNN	val-picked
random labels	random	10	0.711	0.666	0.763
random labels	time	10	0.724	0.730	0.771
k-means clusters	random	8	0.786	0.767	0.795
k-means clusters	time	8	0.764	0.748	0.778
real labels	random	10	0.665	0.797	0.802
real labels	time	10	0.762	0.802	0.785

With real labels, the last layer moves from 0.665 to 0.762 over 10 runs. Any layer study of an in-context model should keep probe rows out of the context's period, or it will find that the last layers “forget” when they have only specialised.

4.6 Retrieval: frozen embeddings are not recommendation embeddings

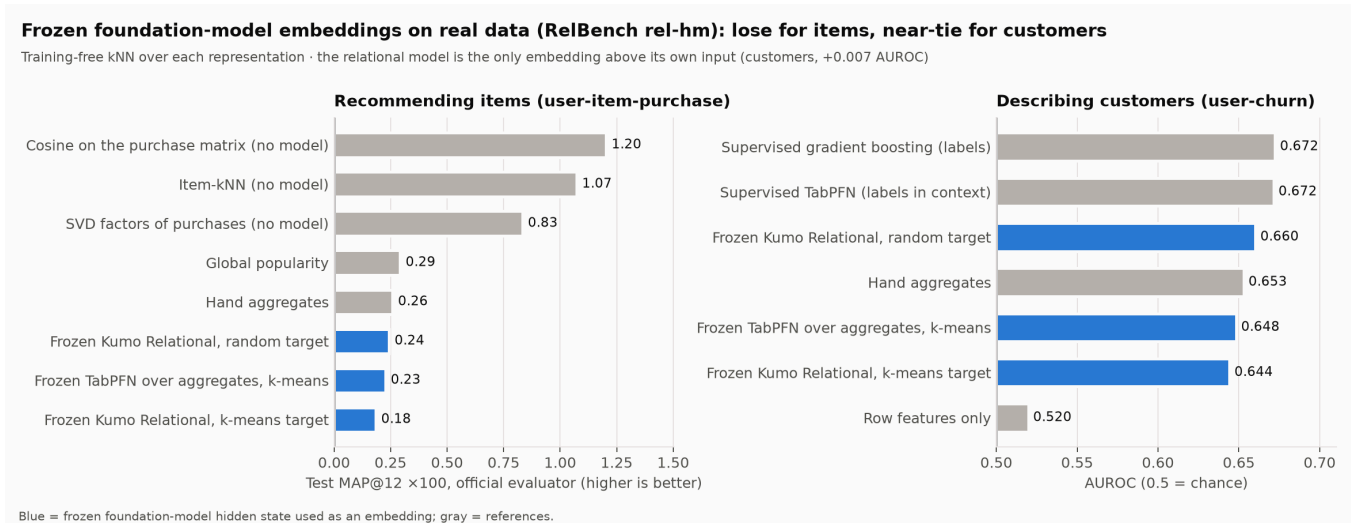


Figure 4. RelBench rel-hm. Left: user-item-purchase, test MAP@12 (×100), training-free kNN over each customer representation. Right: user-churn, kNN-probe AUROC. Blue: a frozen model's hidden state as the embedding.

On a synthetic shop database with planted segments, TabPFN's hidden state with k-means labels in context recovers the segments (P@10 0.82; a GNN trained on the database 0.88). On rel-hm, every frozen embedding (MAP@12 ×100 0.18–0.24) is below hand aggregates (0.26) and far below cosine similarity on the purchase matrix (1.20). On TEmBed's text-heavy Wikidata triplets TabPFN rows score 0.55–0.61 against 0.77 for a small text encoder, whatever the in-context labels. Frozen hidden states summarise an entity's features; for item retrieval the interaction matrix is the right representation.

5. Discussion and limitations

What to use. For an entity embedding from a frozen model on relational data: take Kumo Relational's graph layer, with random labels if no labels exist; with labels, probe the graph layer rather than trusting the head. For TabPFN with labels, use blocks 8–9. Without validation labels, prefer the layer with the lowest intrinsic dimension over the last layer.

Limitations. One seed and one context size per model; 4,000 probe-training rows; linear and kNN probes only. Several tasks are small (rel-f1 top3: 677 probe rows; rel-event repeat: 246 test rows), so per-task differences under 0.02 are noise; we report means over 8 tasks. The time split puts rel-f1's context before 1987, which

lowers every score there equally. The flattened features drop text columns, which helps Kumo Relational on rel-trial. The six-model study (J21) covers 3 of its 5 tasks in this draft.

Conclusion. The best embedding in a frozen tabular or relational foundation model is usually not its last layer. The last layers converge on the in-context label; the earlier layers keep the entity. A label-free statistic can find a good layer, and a simple time split is needed to see any of this.

References

- Robinson, J. et al. (2024). RelBench: A Benchmark for Deep Learning on Relational Databases. NeurIPS Datasets and Benchmarks.
- Hollmann, N. et al. (2025). Accurate predictions on small data with a tabular foundation model. Nature.
- Qu, J. et al. (2025). TabICL: A Tabular Foundation Model for In-Context Learning on Large Data.
- NVIDIA (2026). Structured Data Models: Kumo Tabular and Kumo Relational (open weights).
- IBM Research (2026). Towards Universal Tabular Embeddings: A Benchmark Across Data Tasks (TEmBed). arXiv:2604.21696.
- Is One Layer Enough? Depth Redundancy in Tabular Foundation Models (2026). ICML.
- A Mechanistic Study of Tabular Foundation Models (2026). arXiv preprint.
- Alain, G. & Bengio, Y. (2017). Understanding intermediate layers using linear classifier probes. ICLR Workshop.
- Kornblith, S. et al. (2019). Similarity of Neural Network Representations Revisited. ICML.
- Garrido, Q. et al. (2023). RankMe: Assessing the Downstream Performance of Pretrained Self-Supervised Representations by Their Rank. ICML.
- Facco, E. et al. (2017). Estimating the intrinsic dimension of datasets by a minimal neighborhood information. Scientific Reports.
- Ethayarajh, K. (2019). How Contextual are Contextualized Word Representations? EMNLP.
- Experiments and this paper were produced with Claude Code; tables and the numbers in the text are filled from the run directories by `paper/build_paper.py`.